



FACULTÉ
**GESTION, ÉCONOMIE,
SCIENCES**
Université Catholique
de Lille 1875

Livrable 3 – Projet Master

Master Cyber – Chevalier Enzo

Table des matières

Contexte et objectif.....	3
Description du projet.....	3
Objectif du projet	3
Analyse de l'existant.....	4
Cahier des charges	5
Exigences fonctionnelles.....	5
Exigences non fonctionnelles.....	7
Sécurité.....	7
Performance	7
Maintenabilité.....	7
Testabilité.....	8
Méthodologie & Organisation du travail	9
Méthodologie	9
Organisation du travail	9
Planning (Gantt)	10
Conception de la solution.....	11
Choix des technologies.....	11
Architecture	11
Modèles de données	12
Modèles d'utilisation.....	13
Conclusions.....	15
Problèmes rencontrés	15
Améliorations futures.....	15
Annexes.....	16
Annexe A – Liens utiles du projet.....	16
Annexe B – Identifiants de connexion en tant qu'admin.....	16

Contexte et objectif

Description du projet

Dans le cadre de mon cursus académique, j'ai eu l'occasion de réaliser régulièrement des projets techniques dans différents domaines tels que le développement web et le jeu vidéo.

Tous ces projets sont généralement stockés sur des plateformes comme GitHub, accompagnés d'une documentation plus ou moins structurée (README, notes personnelles, etc.).

Cependant, avec les années, je me suis rendu compte que revenir sur d'anciens projets était compliqué, principalement sur l'aspect technique. Régulièrement, je me posais les mêmes questions : *Quelles sont les commandes pour lancer le projet ? Sur quelles plateformes chaque partie est-elle hébergée ?*

Tout cela rend donc leur consultation, leur mise à jour et leur exploitation plus complexes.

De plus, les solutions classiques de présentation de projets, comme les portfolios en ligne, sont principalement orientées vers une présentation visuelle, sans réellement prendre en compte les aspects techniques des projets.

Objectif du projet

C'est pourquoi l'objectif de ce projet est de concevoir et développer une plateforme web personnelle permettant de :

- Centraliser l'ensemble de mes projets techniques,
- Structurer les informations associées (description, stack technique, exécution, état actuel, etc.),
- Mettre en place une distinction claire entre les données que je souhaite rendre publiques et celles que je souhaite conserver privées, tout en permettant de modifier dynamiquement leur visibilité,
- Offrir une vue d'ensemble de mes projets, notamment en permettant de connaître leur dernière mise à jour grâce à l'automatisation de la récupération d'informations depuis des sources externes, principalement GitHub,
- Proposer une interface claire et professionnelle destinée aux recruteurs potentiels.

Ce projet vise ainsi à répondre à un double besoin :

- Un outil de gestion technique personnel,
- Un portfolio dynamique et pertinent pour la présentation de mes compétences.

Analyse de l'existant

Aujourd'hui, plusieurs solutions permettent de créer rapidement un portfolio en ligne, comme Wix, Squarespace, Webflow ou Portfoliobox. Ces plateformes proposent des interfaces simples, souvent basées sur des templates ou des outils no-code, afin de présenter visuellement des projets, des compétences ou un profil professionnel sans nécessiter de compétences techniques avancées.

Cependant, ces solutions restent principalement orientées vers la mise en forme visuelle et la présentation statique. Elles répondent moins bien à mon besoin, qui vise à gérer des projets techniques de manière plus complète : centralisation des informations, documentation d'exécution, distinction entre données publiques et privées, automatisation via GitHub et suivi technique. Ces limites justifient donc le développement d'une solution personnalisée, davantage centrée sur la gestion technique des projets que sur la simple présentation graphique.

Cahier des charges

Exigences fonctionnelles

Les exigences fonctionnelles définissent les fonctionnalités attendues de l'application et permettent de préciser son comportement du point de vue utilisateur. Dans le cadre de ce projet, elles concernent à la fois la consultation publique du portfolio et la gestion privée des projets par l'administrateur. Le tableau suivant synthétise les principales fonctionnalités intégrées à l'application et leur description.

Fonctionnalité	Description
Page publique du portfolio	Affichage du profil, des liens publics, de l'avatar, du parcours et des projets publics.
Authentification admin	Connexion à l'espace privé via pseudo ou e-mail + mot de passe, avec NextAuth.
Sécurité du compte admin	Table User, rôle admin, mot de passe stocké sous forme hashée avec bcrypt.
Gestion du profil public	Modification du nom affiché, titre, bio, e-mail, GitHub, LinkedIn et avatar.
Gestion du parcours	Ajout, modification, suppression et affichage public des formations, expériences et certifications.
Gestion manuelle des projets	Création, modification, suppression, statut, visibilité, liens GitHub/démo, dates et descriptions.
Détail public d'un projet	Page publique /projects/[slug] affichant les informations détaillées d'un projet public.
Tags projets	Ajout de tags aux projets, affichage public et filtrage des projets sur la page d'accueil.

Gestion fine de la visibilité	Visibilité globale publique/privée sur un projet. Ainsi que la visibilité publique/privée de certaines sections d'un projet.
Déploiement préproduction	Déploiement automatique sur la VM de préproduction après CI réussie sur dev.
Healthcheck	Route /api/health utilisée pour vérifier l'état de l'application.
Déploiement production	Déploiement automatique sur la VM de production après CI réussie sur main.
Déploiement zéro downtime	Stratégie prévue pour la production avec healthcheck avant bascule.
Import GitHub	Création d'un projet à partir d'un lien GitHub.
Synchronisation GitHub	Mise à jour automatique des métadonnées depuis l'API GitHub.
Runbook projet	Documentation des commandes d'installation, lancement, maintenance et debug d'un projet.
Exécution depuis runbook	Bouton permettant d'exécuter une action définie dans un runbook.
Logs d'activité admin	Historique des actions importantes réalisées dans l'administration.
Statistiques publiques	Page ou section statistiques pour les visiteurs. (Langages utilisés en %)
Monitoring production	Mise en place de métriques, alerting et visualisation Grafana.

Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les contraintes techniques et qualitatives que doit respecter le système. Elles ne décrivent pas les fonctionnalités, mais les conditions dans lesquelles celles-ci doivent fonctionner (performance, sécurité, maintenabilité, etc.). Je vais donc définir ci-dessous une liste des différentes exigences non fonctionnelles, avec une description ainsi qu'une liste des attendus.

Sécurité

Le système doit garantir la protection des données et des accès.

- Les mots de passe doivent être stockés sous forme hashée.
- Les routes sensibles doivent être protégées par authentification.
- Les accès non autorisés doivent être bloqués.
- Un système de rate limiting doit être appliqué sur les routes critiques (login).
- Les données privées ne doivent jamais être exposées côté public.
- Les communications doivent être sécurisées (HTTPS en production et préproduction via Cloudflare).
- Les sessions doivent être sécurisées (cookies sécurisés, expiration).

Performance

Le système doit être rapide et fluide pour l'utilisateur.

- Le temps de chargement des pages doit être optimisé.
- Les requêtes à la base de données doivent être efficaces.
- Les données externes (GitHub) doivent être mises en cache si nécessaire.
- L'application doit supporter plusieurs requêtes simultanées sans ralentissement notable.
- Les ressources statiques doivent être optimisées (images, scripts).

Maintenabilité

Le système doit être facilement modifiable et évolutif.

- Le code doit être structuré et organisé (architecture claire).
- Le projet doit utiliser TypeScript avec typage strict.
- Les bonnes pratiques de développement doivent être respectées.
- Les fonctions doivent être modulaires et réutilisables.
- La documentation technique doit être fournie.
- Les conventions de code (linting, formatage) doivent être respectées.

Testabilité

Le système doit pouvoir être testé efficacement.

- Des tests unitaires doivent être mis en place.
- Des tests de composants doivent être réalisés.
- Des tests end-to-end doivent être implémentés.
- Les tests doivent être exécutés automatiquement via CI/CD.
- Les fonctionnalités critiques doivent être couvertes par des tests.

Méthodologie & Organisation du travail

Afin d'assurer une gestion efficace du projet et de respecter les délais imposés, une organisation structurée a été mise en place. Le projet est réalisé selon une approche inspirée des méthodes Agile, adaptée à un contexte de développement individuel.

Méthodologie

Ce projet repose sur une méthodologie agile simplifiée, combinée à un tableau Kanban. Le travail est découpé en tickets, chacun correspondant à une évolution concrète de l'application. Les tickets sont suivis selon leur état d'avancement afin de prioriser les tâches, limiter leur périmètre et conserver une progression régulière. Ils sont remplis sur un Github project, et ils seront rangés selon le schéma suivant :



Organisation du travail

L'organisation du travail comprend la manière dont je vais mettre en place le projet, ainsi les bonnes conventions et habitudes à prendre pour garantir une bonne fluidité dans mon travail. En tout premier pour assurer un système de versionning et d'environnement de production/pré-production, le projet sera hébergé sur GitHub avec la stratégie suivante :

Plusieurs branches seront mises en place :

- Main : version stable (production)
- Develop : version en cours de développement (pré-production)
- Feature : dédiées à chaque nouvelle fonctionnalité
- Hotfix : correction urgente sur l'environnement de production

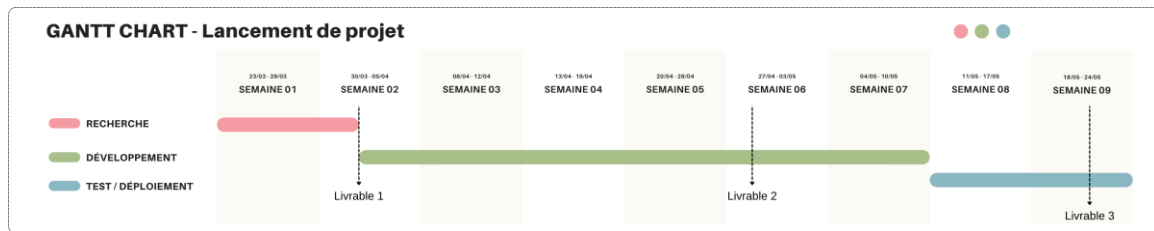
Chaque fonctionnalité est développée dans une branche dédiée, puis intégrée via une Pull Request.

Ensuite pour assurer la qualité du code je vais suivre plusieurs conventions, telles que conventional commits pour structurer les messages de commit. Ainsi que le respect des règles de linting et formatage. Au-delà des conventions de code, une CI/CD avec GitHub Actions est mise en place afin d'exécuter automatiquement les vérifications, les tests et le

build. La branche dev déclenche le déploiement automatisé de la préproduction après une CI réussie. La branche main sera utilisée pour la production et déclenchera un déploiement automatisé avec une stratégie blue/green afin de garantir une mise en ligne sans interruption de service.

Pour suivre le rythme des sprints hebdomadaires mon travail sera organisé en plusieurs sessions de développement régulières dans la semaine.

Planning (Gantt)



Conception de la solution

Cette partie décrit les choix techniques et l'architecture du système mis en place pour répondre aux besoins définis précédemment. Elle vise à justifier les décisions prises en termes de technologies, d'organisation du système et de modélisation des données.

Choix des technologies

Les technologies utilisées ont été sélectionnées en fonction de leur modernité, de leur pertinence vis-à-vis des besoins du projet, ainsi que de mes expériences personnelles et des recherches effectuées.

Le projet repose sur Next.js avec TypeScript, afin de regrouper le front-end et la logique serveur dans une même application tout en bénéficiant d'un typage strict et d'une structure maintenable. L'authentification est assurée par NextAuth, qui permet de gérer les sessions utilisateur et de protéger les parties privées de l'application sans réimplémenter un système complet d'authentification.

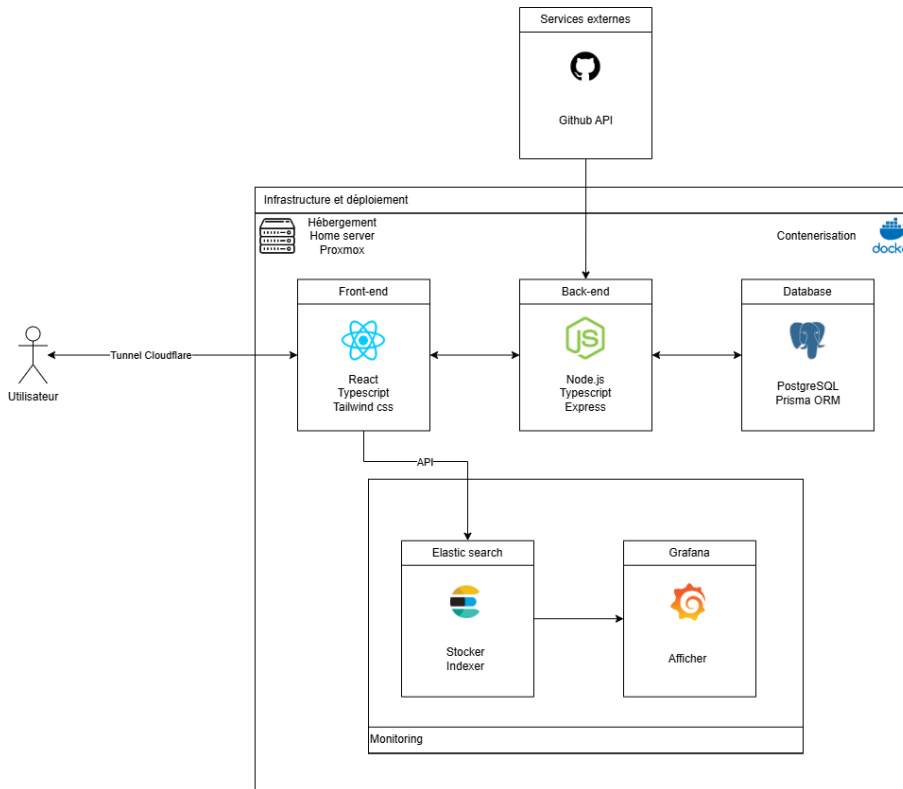
Les données sont stockées dans PostgreSQL et manipulées via Prisma, afin de bénéficier d'une base relationnelle robuste et d'un accès typé aux données. L'application est conteneurisée avec Docker et Docker Compose pour garantir un environnement reproductible entre le développement, la préproduction et la production. GitHub Actions est utilisé pour automatiser les tests, les vérifications et le déploiement.

Architecture

Plus précisément, la préproduction est hébergée sur une VM Debian dédiée dans Proxmox VE. Elle utilise Docker Compose, PostgreSQL, l'application Next.js et Cloudflare Tunnel. Le déploiement est automatisé par GitHub Actions grâce à un runner auto-hébergé installé sur la VM. Le workflow de déploiement récupère la branche dev, reconstruit les conteneurs, applique les migrations Prisma et recrée le compte admin si nécessaire.

La production sera hébergée sur une VM séparée. L'objectif est de mettre en place un déploiement sans interruption perceptible pour l'utilisateur (zero downtime) avec un reverse proxy interne. La stratégie envisagée consiste à lancer une nouvelle version de l'application en parallèle de la version active, à vérifier son état via `/api/health`, puis à basculer le trafic vers la nouvelle instance uniquement si elle répond correctement. En cas d'échec, l'ancienne version reste disponible et le déploiement peut être annulé.

Enfin, le monitoring sera également hébergé sur une VM séparée afin d'isoler les outils d'observabilité du reste de l'application. Cette VM pourra accueillir Grafana ainsi que les services nécessaires à la collecte et à la visualisation des métriques. Cette séparation permettra de surveiller la production sans ajouter de charge directe sur la VM applicative.



Modèles de données

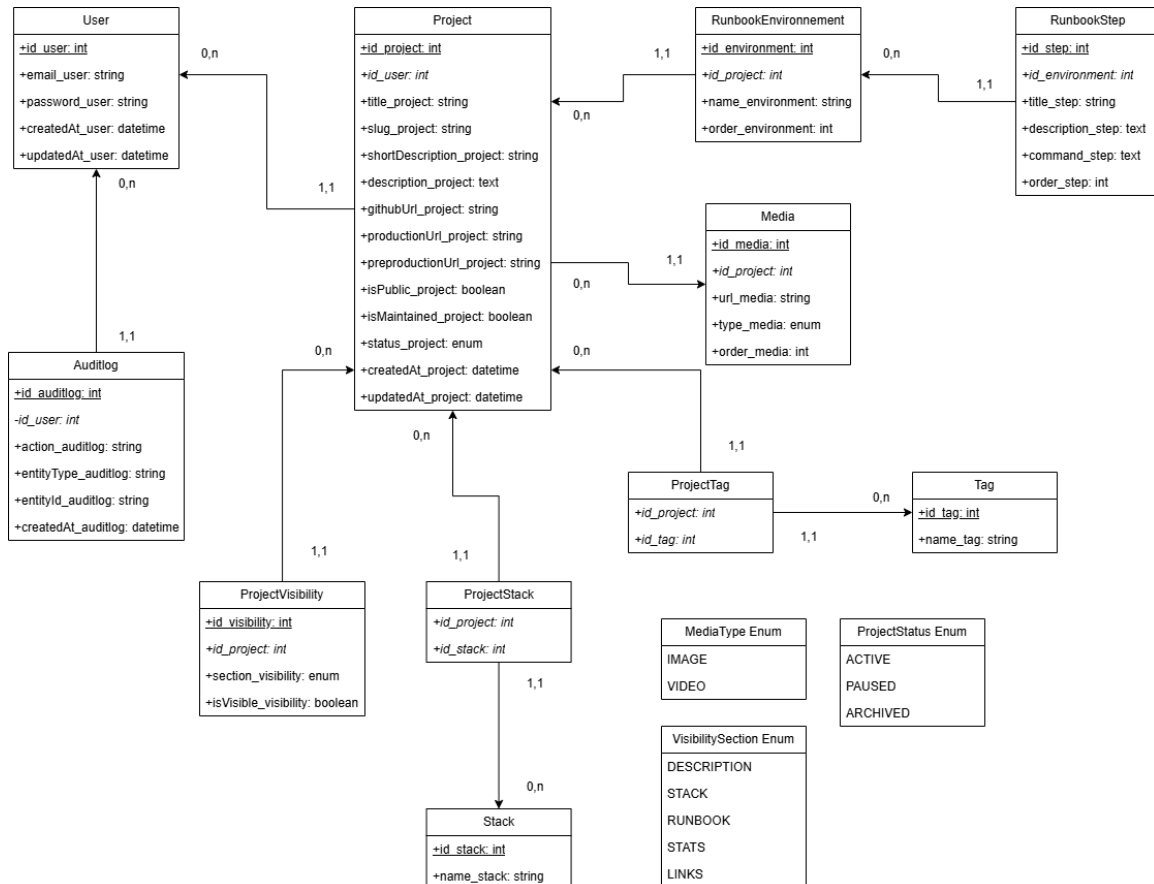
Le modèle de données de l'application est centré autour de l'entité **Project**, qui représente les projets techniques. Chaque projet est associé à un utilisateur (**User**) et contient des informations descriptives, des liens et des métadonnées (statut, visibilité, maintenance).

La classification des projets repose sur deux éléments : les tags (**Tag**) et les technologies (**Stack**), liés aux projets via des relations plusieurs-à-plusieurs (**ProjectTag** et **ProjectStack**).

Un système de runbook permet de documenter l'exécution des projets, avec des environnements (**RunbookEnvironment**) contenant des étapes (**RunbookStep**). Les médias (**Media**) permettent d'illustrer les projets.

La visibilité des différentes sections d'un projet est gérée via ProjectVisibility, offrant une distinction entre données publiques et privées. Enfin, les actions importantes sont tracées dans AuditLog.

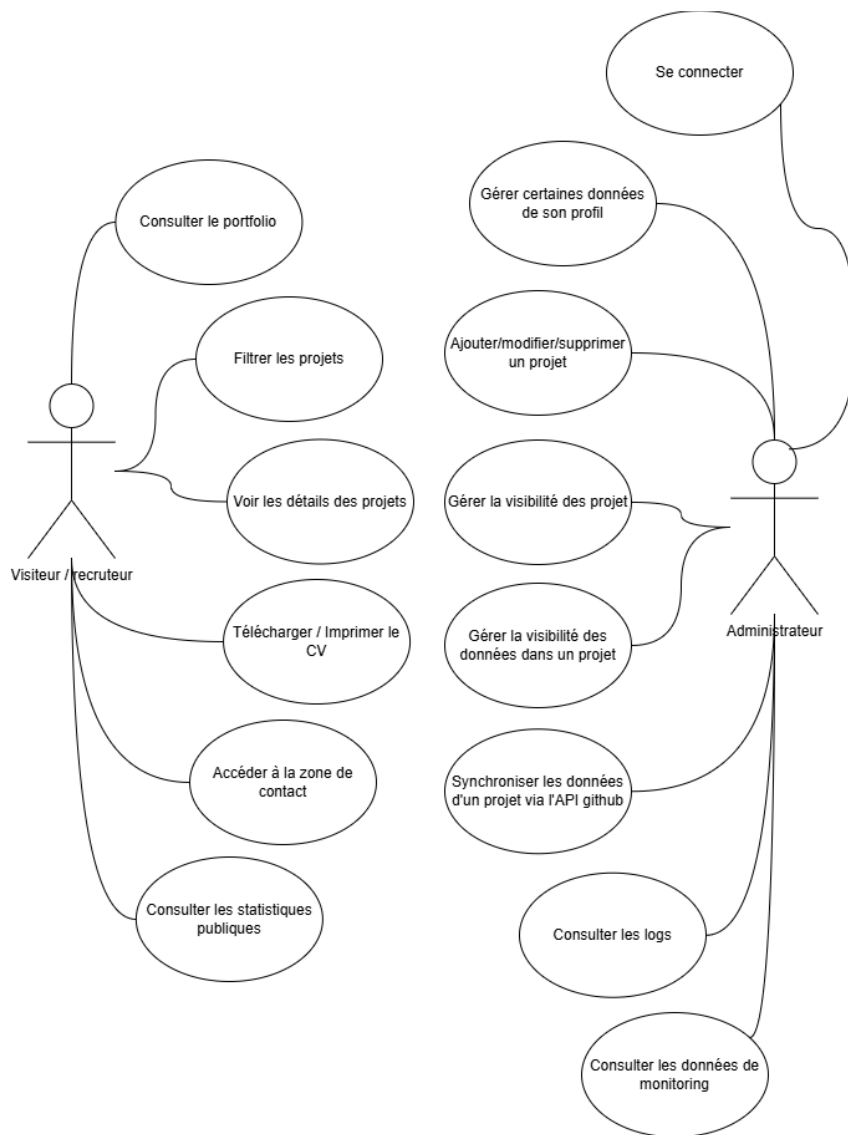
Des types énumérés (enums) sont utilisés pour garantir la cohérence des données, notamment pour le statut des projets, les types de médias et les sections de visibilité.



Modèles d'utilisation

Le modèle d'utilisation présente les interactions entre les différents acteurs et le système. Il met en évidence les principales fonctionnalités accessibles aux visiteurs/recruteurs ainsi que celles réservées à l'administrateur.

Le système comporte deux acteurs principaux : le visiteur et l'administrateur. Le visiteur peut consulter le portfolio public, parcourir les projets, filtrer les projets par tags et accéder au détail public d'un projet. L'administrateur, après authentification, peut gérer son profil, son parcours, ses projets, leurs tags, leur visibilité et leurs informations techniques.



Conclusions

Problèmes rencontrés

Plusieurs difficultés ont été rencontrées pendant le projet. La première concerne la gestion du workflow Git : la mise en place des branches, des Pull Requests et des merges vers les bons environnements a nécessité plusieurs ajustements. Ces erreurs ont permis de mieux structurer le workflow entre dev, main, les branches de fonctionnalités et les branches de correction.

La deuxième difficulté concerne l'infrastructure. La mise en place de deux environnements distincts, la configuration de Docker, Cloudflare Tunnel, GitHub Actions et du runner auto-hébergé ont demandé un travail important. Les limites matérielles du serveur Proxmox ont également dû être prises en compte pour dimensionner correctement les VM de préproduction, de production et de monitoring.

La troisième difficulté concerne l'estimation du temps nécessaire à certaines tâches techniques. Certaines fonctionnalités, comme la mise en place d'un déploiement zéro downtime, demandaient plus de temps que prévu car je ne les avais jamais réalisées auparavant. J'ai donc dû ajuster mes priorités et reporter certaines évolutions dans les améliorations futures afin de conserver un périmètre réaliste pour la maintenance.

Améliorations futures

- Améliorer le design
- Ajouter une authentification multi-utilisateur afin de permettre à d'autres profils de gérer leur propre portfolio.
- Générer automatiquement une version PDF du portfolio ou du CV à partir des données publiques.
- Ajouter un historique de versions pour les fiches projets et les runbooks.
- Traduction du portfolio en anglais.

Annexes

Annexe A – Liens utiles du projet

Repository Github : <https://github.com/SiBlue7/projet-master-portfolio>

Project Github : <https://github.com/users/SiBlue7/projects/5>

Site production : <https://portfolio.justdoeat.org>

Site préproduction : <https://preprod.justdoeat.org>

Annexe B – Identifiants de connexion en tant qu'admin

Un compte administrateur temporaire est mis à disposition du jury afin de tester l'espace d'administration. Ces identifiants sont fournis uniquement dans le cadre de l'évaluation et seront modifiés ou supprimés après la soutenance.

Identifiant : admin-temp

Mot de passe : admin-temp